

Exercises 12.1: Backtracking

Questions and Answers

Question 1

For the 4-Queens problem, continue the row-by-row backtracking search after the first solution has been found. Assume columns are tried in increasing order and the solution vector is

$$x = [x[1], x[2], x[3], x[4]],$$

where $x[i]$ is the column chosen for the queen in row i . Find the second solution.

Answer 1

The second solution is

$$x = [3, 1, 4, 2].$$

The first successful branch is

$$\emptyset \rightarrow [2] \rightarrow [2, 4] \rightarrow [2, 4, 1] \rightarrow [2, 4, 1, 3].$$

After printing this solution, backtracking returns to earlier rows and continues trying the next safe choices. The next successful branch is

$$\emptyset \rightarrow [3] \rightarrow [3, 1] \rightarrow [3, 1, 4] \rightarrow [3, 1, 4, 2].$$

The queen positions are

$$(1, 3), (2, 1), (3, 4), (4, 2).$$

Check the safety conditions:

columns 3, 1, 4, 2 are all distinct,

and for every pair of rows $i < k$,

$$|x[i] - x[k]| \neq |i - k|.$$

So no two queens share a row, column, or diagonal. Therefore the second solution is

$$[3, 1, 4, 2].$$

Question 2

Explain how the board's symmetry can be used to find the second solution to the four-queens problem.

Answer 2

The second solution is the mirror image of the first solution.

For an $n \times n$ board, reflection across the vertical middle line changes every column c to

$$c' = n + 1 - c.$$

For $n = 4$, the first solution is

$$[2, 4, 1, 3].$$

Reflecting it gives

$$[5 - 2, 5 - 4, 5 - 1, 5 - 3] = [3, 1, 4, 2].$$

Thus the board's symmetry gives the second solution immediately:

$$[3, 1, 4, 2].$$

Question 3

Which is the last solution to the five-queens problem found by the backtracking algorithm?

Answer 3

Assuming row-by-row backtracking and trying columns in increasing order, the last five-queens solution found is

$$[5, 3, 1, 4, 2].$$

That means the queens are placed at

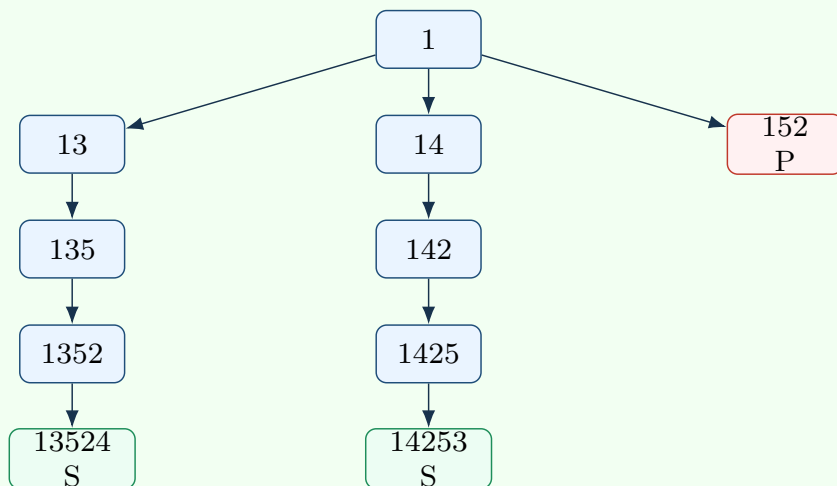
$$(1, 5), (2, 3), (3, 1), (4, 4), (5, 2).$$

The full backtracking search finds the following solutions in this order:

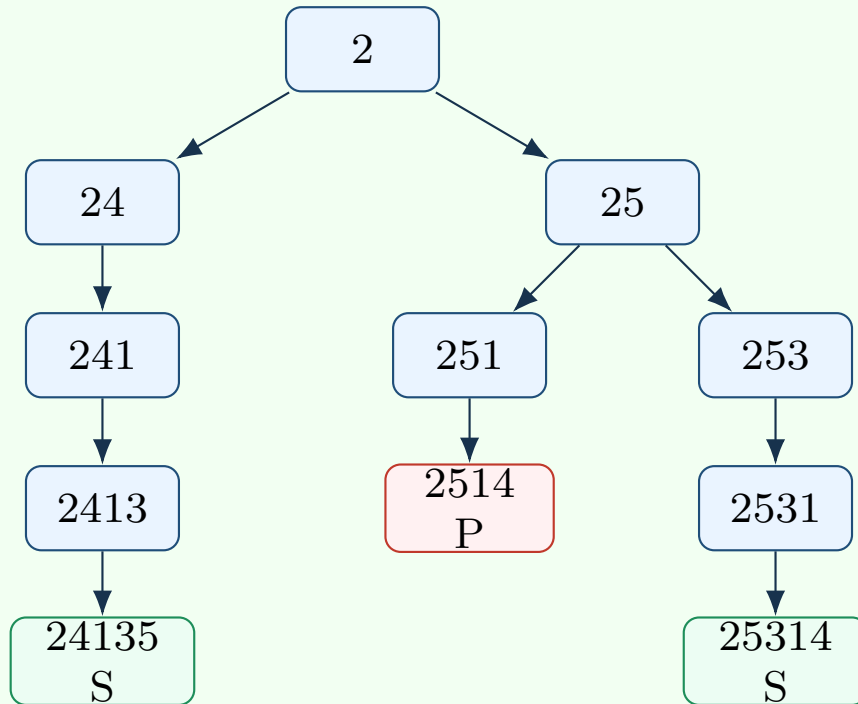
No.	Solution vector x	Queen positions
1	[1, 3, 5, 2, 4]	(1, 1), (2, 3), (3, 5), (4, 2), (5, 4)
2	[1, 4, 2, 5, 3]	(1, 1), (2, 4), (3, 2), (4, 5), (5, 3)
3	[2, 4, 1, 3, 5]	(1, 2), (2, 4), (3, 1), (4, 3), (5, 5)
4	[2, 5, 3, 1, 4]	(1, 2), (2, 5), (3, 3), (4, 1), (5, 4)
5	[3, 1, 4, 2, 5]	(1, 3), (2, 1), (3, 4), (4, 2), (5, 5)
6	[3, 5, 2, 4, 1]	(1, 3), (2, 5), (3, 2), (4, 4), (5, 1)
7	[4, 1, 3, 5, 2]	(1, 4), (2, 1), (3, 3), (4, 5), (5, 2)
8	[4, 2, 5, 3, 1]	(1, 4), (2, 2), (3, 5), (4, 3), (5, 1)
9	[5, 2, 4, 1, 3]	(1, 5), (2, 2), (3, 4), (4, 1), (5, 3)
10	[5, 3, 1, 4, 2]	(1, 5), (2, 3), (3, 1), (4, 4), (5, 2)

The full safe-prefix backtracking tree is shown below. Compact labels are used: for example, 13524 means [1, 3, 5, 2, 4]. A green node marked S is a complete solution, and a red node marked P is a dead-end prefix.

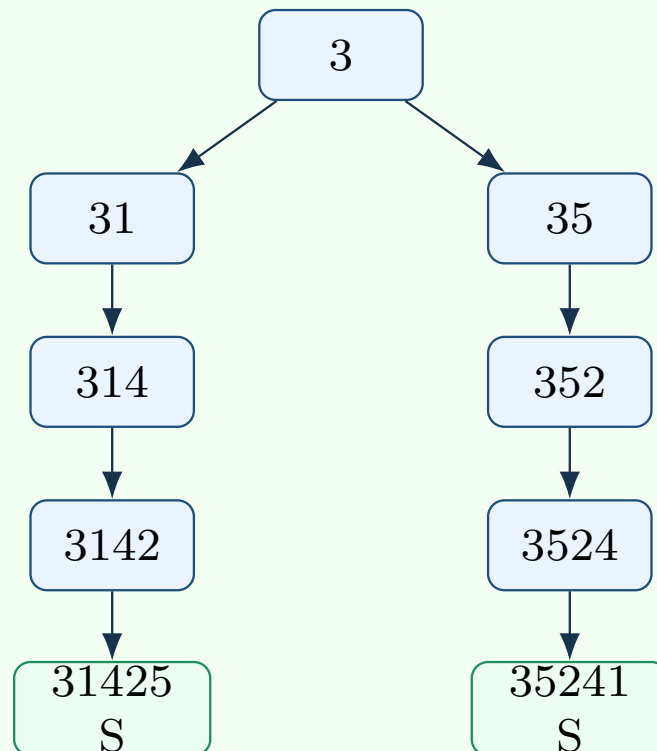
Subtree Starting with Row 1 Column 1



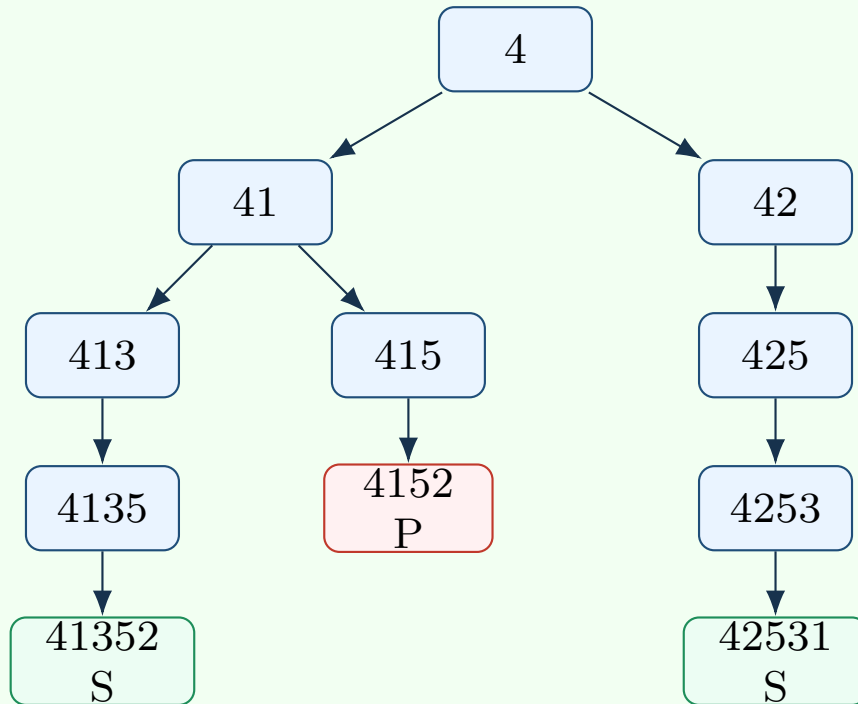
Subtree Starting with Row 1 Column 2



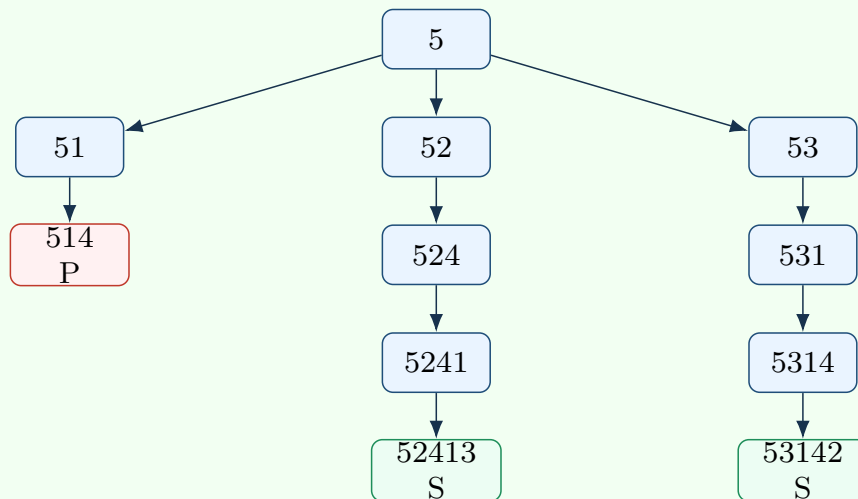
Subtree Starting with Row 1 Column 3



Subtree Starting with Row 1 Column 4



Subtree Starting with Row 1 Column 5



Each row is produced by choosing a safe column for row 1, then row 2, and so on until all five rows are filled. A partial vector is rejected as soon as it violates

$$x[i] \neq x[k] \quad \text{or} \quad |x[i] - x[k]| \neq |i - k|.$$

Therefore the last solution found by the complete backtracking search is

$$[5, 3, 1, 4, 2].$$

Question 4

Use the board's symmetry to find at least four other solutions to the five-queens problem.

Answer 4

Take the five-queens solution

$$x = [5, 3, 1, 4, 2].$$

A solution vector means that the queen in row i is placed in column $x[i]$. Thus the queen positions are

$$(1, 5), (2, 3), (3, 1), (4, 4), (5, 2).$$

Since $n = 5$, every reflection or rotation uses $n + 1 = 6$. Board symmetry preserves row distances, column distances, and diagonal distances, so every transformed arrangement is also a valid solution.

First use vertical reflection. This changes every column c into $6 - c$:

$$\begin{aligned} [5, 3, 1, 4, 2] &\mapsto [6 - 5, 6 - 3, 6 - 1, 6 - 4, 6 - 2] \\ &= [1, 3, 5, 2, 4]. \end{aligned}$$

Now use horizontal reflection. This changes row r into $6 - r$, so the row order is reversed:

$$[(1, 5), (2, 3), (3, 1), (4, 4), (5, 2)] \mapsto [(5, 5), (4, 3), (3, 1), (2, 4), (1, 2)].$$

Writing the columns in row order 1, 2, 3, 4, 5, we get

$$[2, 4, 1, 3, 5].$$

For rotation by 90° , use

$$(r, c) \mapsto (c, 6 - r).$$

Then

$$\begin{aligned} (1, 5) &\mapsto (5, 5), & (2, 3) &\mapsto (3, 4), & (3, 1) &\mapsto (1, 3), \\ (4, 4) &\mapsto (4, 2), & (5, 2) &\mapsto (2, 1). \end{aligned}$$

After sorting by row number, the positions are

$$(1, 3), (2, 1), (3, 4), (4, 2), (5, 5),$$

so the solution vector is

$$[3, 1, 4, 2, 5].$$

For rotation by 180° , use

$$(r, c) \mapsto (6 - r, 6 - c).$$

Then

$$\begin{aligned} (1, 5) &\mapsto (5, 1), & (2, 3) &\mapsto (4, 3), & (3, 1) &\mapsto (3, 5), \\ (4, 4) &\mapsto (2, 2), & (5, 2) &\mapsto (1, 4). \end{aligned}$$

After sorting by row number, the positions are

$$(1, 4), (2, 2), (3, 5), (4, 3), (5, 1),$$

so the solution vector is

$$[4, 2, 5, 3, 1].$$

Therefore, at least four other solutions are

$$\boxed{[1, 3, 5, 2, 4], \quad [2, 4, 1, 3, 5], \quad [3, 1, 4, 2, 5], \quad [4, 2, 5, 3, 1].}$$

Question 5

Implement the backtracking algorithm for the n -queens problem in the language of your choice. Run your program for a sample of n values to get the numbers of nodes in the algorithm's state-space trees. Compare these numbers with the numbers of candidate solutions generated by the exhaustive-search algorithm for this problem.

Answer 5

A direct implementation is:

```
count = 0
x = []

def safe(row, col):
    for r in range(row):
        if x[r] == col:
            return False
        if abs(x[r] - col) == abs(r - row):
            return False
    return True

def queens(row, n):
    global count
    count += 1
    if row == n:
        return
    for col in range(n):
        if safe(row, col):
            x.append(col)
            queens(row + 1, n)
            x.pop()
```

Counting each recursive call as one state-space-tree node gives:

n	Backtracking nodes	Exhaustive n^n	candidates	Saving factor
4	17	256		about 15.1 times
5	54	3125		about 57.9 times
6	153	46656		about 305 times
7	552	823543		about 1492 times
8	2057	16777216		about 8156 times

Backtracking is much smaller because it rejects a partial placement as soon as it creates a column or diagonal conflict.

Question 6

For each value of n for which you run your program in the previous question, estimate the size of the state-space tree by the random-path method and compare the estimate with the actual number of nodes you obtained.

Answer 6

Knuth's random-path method estimates the number of nodes without building the whole state-

space tree. Along one randomly chosen root-to-leaf path, let

$$c_1, c_2, \dots, c_m,$$

be the numbers of promising children at successive levels. The estimate from that one path is

$$T = 1 + c_1 + c_1 c_2 + c_1 c_2 c_3 + \dots + c_1 c_2 \dots c_m.$$

Here, the initial 1 counts the root node.

For the n -queens problem, at each level we count the safe columns for the next row, randomly choose one of those safe columns, and continue until a solution or a dead end is reached.

The following are structured one-path calculations for the same values of n used in the previous question. Each row shows one sampled path, its branching counts, and the resulting estimate T .

n	$(c_1, c_2, \dots, c_m)$	Calculation of T
4	(4, 1, 1, 1)	$T = 1 + 4 + 4 + 4 + 4 = 17$
5	(5, 3, 1, 1, 1)	$T = 1 + 5 + 15 + 15 + 15 + 15 = 66$
6	(6, 4, 1, 1, 1)	$T = 1 + 6 + 24 + 24 + 24 + 24 = 103$
7	(7, 4, 3, 1, 1, 1)	$T = 1 + 7 + 28 + 84 + 84 + 84 + 84 = 456$
8	(8, 5, 3, 3, 2, 1, 1)	$T = 1 + 8 + 40 + 120 + 360 + 720 + 720 + 720 = 2689$

For example, in the $n = 8$ row, the partial products are

$$\begin{aligned} 8, \quad 8 \cdot 5 = 40, \quad 8 \cdot 5 \cdot 3 = 120, \quad 8 \cdot 5 \cdot 3 \cdot 3 = 360, \\ 8 \cdot 5 \cdot 3 \cdot 3 \cdot 2 = 720, \quad 8 \cdot 5 \cdot 3 \cdot 3 \cdot 2 \cdot 1 = 720, \quad 8 \cdot 5 \cdot 3 \cdot 3 \cdot 2 \cdot 1 \cdot 1 = 720. \end{aligned}$$

Adding the root node gives

$$T = 1 + 8 + 40 + 120 + 360 + 720 + 720 + 720 = 2689.$$

Comparison with the actual node counts:

n	Actual nodes	One-path estimate T	Difference	Comment
4	17	17	0	Exact for this path
5	54	66	+12	Overestimate
6	153	103	-50	Underestimate
7	552	456	-96	Underestimate
8	2057	2689	+632	Overestimate

The method is useful because it avoids expanding the full tree. However, a single estimate may be too high or too low because different branches of the n -queens tree can have very different sizes.

The procedure is:

1. Start at row 1.
2. Count how many safe columns are available. This is c_1 .
3. Randomly choose one safe column.
4. Continue to row 2, count the safe choices c_2 , choose one at random, and repeat.
5. Compute the estimate from the product formula.
6. Repeat several times and average the estimates.

Question 7

Design a linear-time algorithm that finds a solution to the n -queens problem for any $n \geq 4$.

Answer 7

A constructive $O(n)$ method is to output the queen columns directly. Let

$$E = (2, 4, 6, \dots) \quad \text{and} \quad O = (1, 3, 5, \dots)$$

be the even and odd column lists up to n .

1. If $n \bmod 6 \notin \{2, 3\}$, output E followed by O .
2. If $n \bmod 6 = 2$, output E , then swap 1 and 3 in O , and move 5 to the end of O .
3. If $n \bmod 6 = 3$, move 2 to the end of E , and move 1 and 3 to the end of O . Then output E followed by O .

Examples:

$$n = 8 : \quad [2, 4, 6, 8, 3, 1, 7, 5],$$

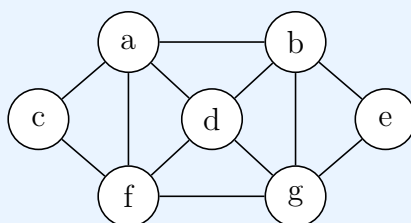
$$n = 9 : \quad [4, 6, 8, 2, 5, 7, 9, 1, 3].$$

The algorithm only creates and prints two lists of total length n , so its running time is

$$\boxed{O(n)}.$$

Question 8

Apply backtracking to the problem of finding a Hamiltonian circuit in the given graph.



Answer 8

Fix the starting vertex as c . The graph has the following adjacency list:

Vertex	Adjacent vertices
a	b, c, d, f
b	a, d, e, g
c	a, f
d	a, b, f, g
e	b, g
f	a, c, d, g
g	b, d, e, f

A Hamiltonian circuit must start at c , visit each of

$$a, b, d, e, f, g$$

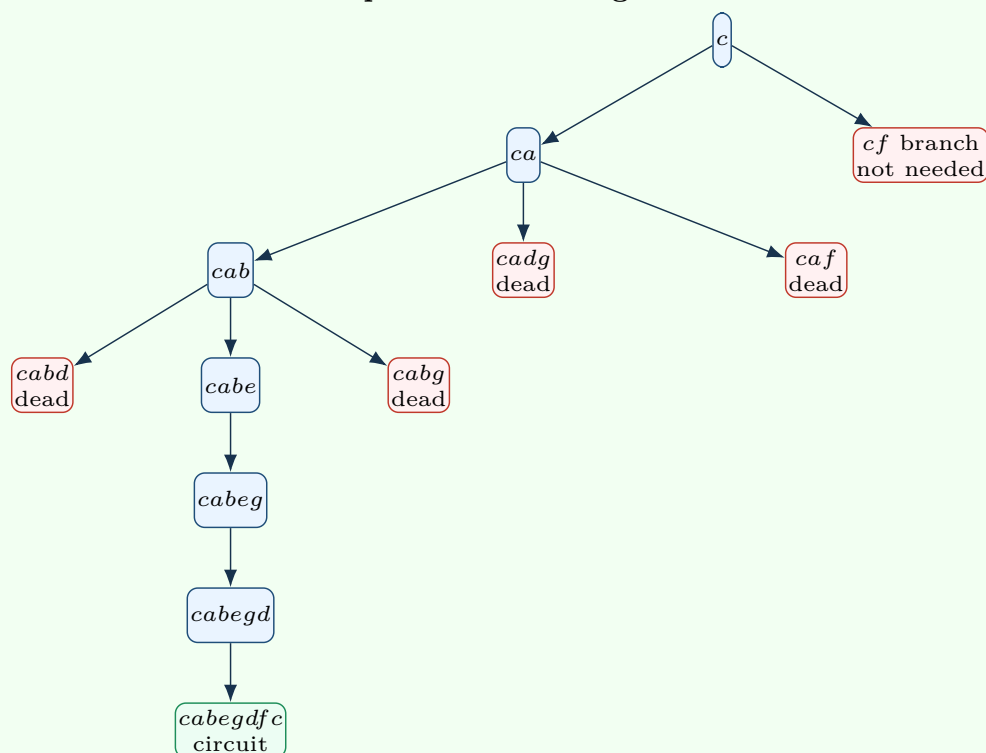
exactly once, and then return to c . Since c is adjacent only to a and f , the circuit must leave c through one of them and return through the other.

Backtracking tries an unused adjacent vertex at each step. We do not need to draw the entire state-space tree; it is enough to show the important dead branches and one successful branch. A label such as *cabegdfc* means

$$c \rightarrow a \rightarrow b \rightarrow e \rightarrow g \rightarrow d \rightarrow f \rightarrow c.$$

Green means a Hamiltonian circuit has been found. Red means the branch is rejected and the algorithm backtracks.

Compact Backtracking Tree



The successful path is therefore

$$c \rightarrow a \rightarrow b \rightarrow e \rightarrow g \rightarrow d \rightarrow f \rightarrow c.$$

Check the edges in this circuit:

$$(c, a), (a, b), (b, e), (e, g), (g, d), (d, f), (f, c),$$

and every one of these edges appears in the graph. Also, the vertices

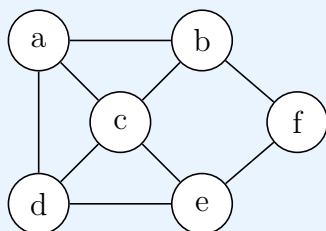
$$a, b, c, d, e, f, g$$

all appear exactly once before returning to *c*. Hence this is a valid Hamiltonian circuit.

Branches are pruned whenever the next vertex is not adjacent to the current vertex, has already been used, or cannot return to *c* after all vertices have been included. This is exactly the backtracking idea: try a choice, reject it when it cannot lead to a circuit, and then return to the previous vertex to try another choice.

Question 9

Apply backtracking to solve the 3-coloring problem for the given graph.



Answer 9

Use the colors R, G, B , and color the vertices in the order

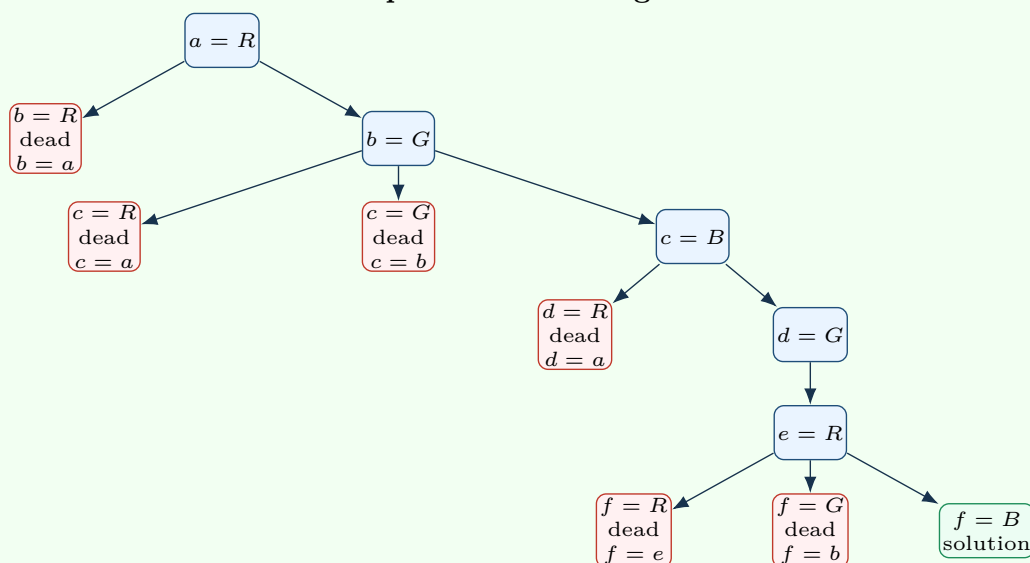
a, b, c, d, e, f .

The graph has the edges

$(a, b), (a, d), (a, c), (d, c), (d, e), (c, b), (c, e), (b, f), (e, f)$.

Backtracking tries colors one vertex at a time and rejects a color if it matches an already-colored adjacent vertex:

Compact Backtracking Tree



The successful branch of this tree is

$a = R, \quad b = G, \quad c = B, \quad d = G, \quad e = R, \quad f = B.$

Vertex	Chosen color	Reason
a	R	first vertex, no earlier constraint
b	G	$b \neq a$
c	B	$c \neq a$ and $c \neq b$
d	G	$d \neq a$ and $d \neq c$
e	R	$e \neq d$ and $e \neq c$
f	B	$f \neq b$ and $f \neq e$

Thus one valid 3-coloring is

$$a = R, \quad b = G, \quad c = B, \quad d = G, \quad e = R, \quad f = B.$$

Check every edge:

$$\begin{aligned} a \neq b, \quad a \neq d, \quad a \neq c, \quad d \neq c, \quad d \neq e, \\ c \neq b, \quad c \neq e, \quad b \neq f, \quad e \neq f. \end{aligned}$$

Every edge joins vertices with different colors, so the coloring is valid.

The graph needs three colors because a, b, c form a triangle. In a triangle, all three vertices are mutually adjacent, so they must receive three different colors.

Question 10

Generate all permutations of $\{1, 2, 3, 4\}$ by backtracking.

Answer 10

Backtracking builds the permutation one position at a time. At each position, choose one unused element.

1234	1243	1324	1342
1423	1432	2134	2143
2314	2341	2413	2431
3124	3142	3214	3241
3412	3421	4123	4132
4213	4231	4312	4321

There are

$$4! = 24$$

permutations.

Question 11

Apply backtracking to solve the subset sum instance $A = \{1, 3, 4, 5\}$ and $d = 11$.

Answer 11

The target is

$$d = 11.$$

The subset

$$\{1, 5, 5\}$$

is not allowed because 5 appears only once. Testing the available elements:

$$1 + 3 + 4 + 5 = 13,$$

$$1 + 3 + 4 = 8, \quad 1 + 3 + 5 = 9, \quad 1 + 4 + 5 = 10, \quad 3 + 4 + 5 = 12.$$

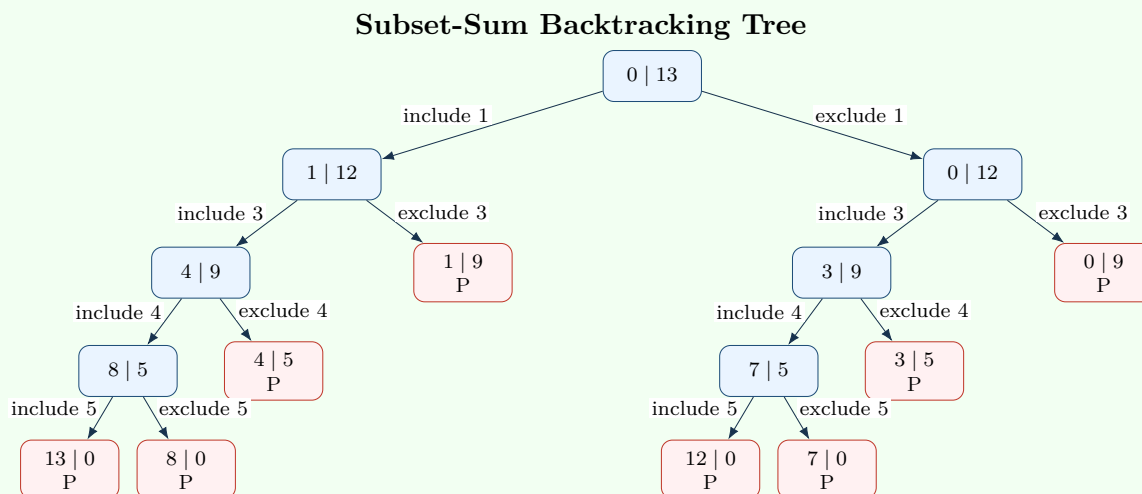
Use the same state notation as the Subset Sum notes:

$$s \mid r,$$

where s is the current selected sum and r is the sum of the remaining undecided items. Here

$$w = (1, 3, 4, 5), \quad m = 11, \quad r_{\text{root}} = 1 + 3 + 4 + 5 = 13.$$

A node is pruned when $s > 11$ or $s + r < 11$. In the tree, the left child includes the next item and the right child excludes it.



The pruned nodes mean:

$$1 \mid 9, 0 \mid 9, 4 \mid 5, 3 \mid 5, 8 \mid 0, 7 \mid 0 \quad \text{have } s + r < 11,$$

while

$$13 \mid 0, 12 \mid 0 \quad \text{have } s > 11.$$

No subset sums to 11. Therefore,

there is no solution.

Backtracking reaches this conclusion by including or excluding each element and pruning branches whose current sum is already too large or whose remaining elements cannot reach the target.

Question 12

In the subset-sum backtracking algorithm, will the algorithm still be correct if we use only one of the two pruning inequalities instead of both?

Answer 12

Yes, the algorithm can still be correct, but it will become less efficient. The important assumption is that the algorithm prints a subset only when the current sum is exactly the target:

$$s = d.$$

The inequalities are pruning tests; they are used to avoid useless branches, not to decide that a subset is a solution.

For sorted positive weights, a node with current sum s , remaining sum r , next item $w[k]$, and target d is usually treated as promising only if

$$s + r \geq d$$

and

$$s + w[k] \leq d.$$

The first inequality,

$$s + r \geq d,$$

means the target is still reachable. If $s + r < d$, then even adding every remaining item is not enough, so the branch cannot contain a solution.

The second inequality,

$$s + w[k] \leq d,$$

prevents an immediate overshoot when the next item is included. Since all weights are positive, once the sum becomes larger than d , adding more items cannot fix it.

Only one test used	What happens
$s + r \geq d$ only	The algorithm may explore branches where including the next item overshoots the target.
$s + w[k] \leq d$ only	The algorithm may explore branches whose remaining items are too small to ever reach the target.

Therefore, using only one inequality does not normally make the answer wrong, but it removes some pruning. The algorithm still finds the correct result because a subset is accepted only when $s = d$. Both inequalities are used together to make the search efficient.

Question 13

The general template for backtracking algorithms works correctly only if no solution is a prefix to another solution to the problem. Change the template's pseudocode to work correctly without this restriction.

Answer 13

The fix is to output a solution when it is found, but not automatically stop expanding that node. If a solution can be a prefix of a longer solution, the algorithm must both process the current solution and continue searching its children when they are promising.

```
Backtrack(x):
    if isSolution(x):
        print x

    if promising(x):
        for each candidate a that can extend x:
            add a to x
            Backtrack(x)
            remove a from x
```

The key change is that `isSolution(x)` is checked separately from `promising(x)`. Finding a solution does not force an immediate return.

Question 14

Write a program implementing a backtracking algorithm for (a) the Hamiltonian circuit problem and (b) the m -coloring problem.

Answer 14

For the Hamiltonian circuit problem:

```
path[0] = 0

def ham(k):
    if k == n:
```

```

    if G[path[n-1]][path[0]]:
        print(path + [path[0]])
        return

    for v in range(1, n):
        if G[path[k-1]][v] and v not in path[:k]:
            path[k] = v
            ham(k + 1)
            path[k] = -1

```

For the m -coloring problem:

```

color = [0] * n

def safe(v, c):
    for u in range(n):
        if G[v][u] and color[u] == c:
            return False
    return True

def color_graph(v):
    if v == n:
        print(color)
        return

    for c in range(1, m + 1):
        if safe(v, c):
            color[v] = c
            color_graph(v + 1)
            color[v] = 0

```

Both programs use the same backtracking structure: choose a candidate, test whether it is safe, recurse, and then undo the choice.